



TUTORIAL INTERACTIVO

GUÍA DEL ESTUDIANTE · NIVEL INTERMEDIO A AVANZADO



TYPESCRIPT

PASO A PASO



Estructuras, POO y Conceptos Avanzados



Concepto · Explicación · Código · Tips · Retos



CONTENIDO DEL TUTORIAL

20 lecciones · 40 retos prácticos

Programación Orientada a Objetos completa

Estructuras avanzadas: Map, Set, Genéricos



LECCIÓN 00

TUTORIAL TYPESCRIPT — PASO A PASO

CÓMO USAR ESTA GUÍA

Sácale el máximo provecho a tu aprendizaje



BIENVENIDO ESTUDIANTE

Este tutorial te llevará paso a paso a través de los conceptos intermedios y avanzados de TypeScript. Cada lección está diseñada con una estructura pedagógica probada: primero entiendes el qué, luego el cómo, después lo ves funcionando, recibes consejos de experto y finalmente lo practicas con dos retos reales. Avanza a tu propio ritmo y vuelve a las lecciones cuantas veces necesites.



ESTRUCTURA DE CADA LECCIÓN

1		CONCEPTO	La definición clara y precisa del tema.
2		EXPLICACIÓN	Desarrollo detallado, paso a paso del funcionamiento.
3		EJEMPLO	Código real con su resultado en consola.
4		TIPS	Consejos de experto para evitar errores comunes.
5		RETOS	Dos ejercicios prácticos para aplicar lo aprendido.



PREPARACIÓN DEL ENTORNO

1. Instala Node.js versión 20 o superior desde nodejs.org
2. Abre la terminal y ejecuta: `npm install -g typescript ts-node`
3. Verifica la instalación con: `tsc --version`
4. Crea una carpeta de trabajo y dentro un archivo con extensión `.ts`
5. Ejecuta tus archivos con: `ts-node nombreArchivo.ts`
6. Instala Visual Studio Code y la extensión oficial de TypeScript



RECOMENDACIONES DE ESTUDIO

- ◆ No avances al siguiente tema hasta resolver los dos retos del actual.

- ✦ Escribe el código a mano en tu editor, no solo lo leas.
- ✦ Modifica los ejemplos para experimentar y entender cómo reaccionan.
- ✦ Si algo falla, lee el error con calma. TypeScript te dirá exactamente qué pasa.
- ✦ Toma notas propias en un cuaderno. La escritura refuerza el aprendizaje.



LECCIÓN 01

TUTORIAL TYPESCRIPT — PASO A PASO

INTERFACES EN TYPESCRIPT

Define la forma que deben tener tus objetos



CONCEPTO

Una interface en TypeScript es un contrato que define qué propiedades y métodos debe tener un objeto. Es como un molde o plantilla que asegura que todos los objetos del mismo tipo sigan la misma estructura, sin proveer la implementación.



EXPLICACIÓN DETALLADA

Las interfaces se declaran con la palabra reservada `interface` seguida de un nombre que por convención inicia con mayúscula. Dentro de las llaves se listan las propiedades con sus tipos. Cuando un objeto se tipa con una interface, el compilador verifica que cumpla exactamente con esa estructura.

Las interfaces pueden tener propiedades obligatorias, propiedades opcionales marcadas con un signo de interrogación (campo?:), propiedades de solo lectura con la palabra `readonly` y también definir la firma de métodos que deben ser implementados.

A diferencia de las clases, las interfaces no generan código JavaScript al compilar; existen únicamente en tiempo de desarrollo para ayudarte a detectar errores. Una de las grandes ventajas de las interfaces es que pueden extenderse entre sí, permitiendo crear jerarquías de contratos reutilizables.



EJEMPLO PRÁCTICO



Caso: Una empresa de mensajería necesita modelar los datos de sus envíos.



CÓDIGO DE EJEMPLO

```
// Sistema de mensajería - Empresa EnvíosYa
interface Direccion {
  ciudad: string;
  barrio: string;
  referencia?: string;    // opcional
}
```

```
interface Envio {
  readonly codigo: string; // no se puede modificar
  remitente: string;
  destinatario: string;
  direccion: Direccion;
  peso: number;
  entregado: boolean;
}

const envio1: Envio = {
  codigo: "EYA-2026-001",
  remitente: "Andrea López",
  destinatario: "Mario Pérez",
  direccion: {
    ciudad: "Bogotá",
    barrio: "Chapinero",
    referencia: "Edificio Azul, apto 502"
  },
  peso: 2.5,
  entregado: false
};

console.log("=== GUÍA DE ENVÍO ===");
console.log("Código:", envio1.codigo);
console.log("Para:", envio1.destinatario);
console.log("Dirección:", envio1.direccion.barrio, "-",
envio1.direccion.ciudad);
console.log("Peso:", envio1.peso, "kg");
console.log("Estado:", envio1.entregado ? "Entregado" : "En tránsito");
```

▶ SALIDA EN CONSOLA

```
=== GUÍA DE ENVÍO ===
Código: EYA-2026-001
Para: Mario Pérez
Dirección: Chapinero - Bogotá
Peso: 2.5 kg
Estado: En tránsito
```

★ TIPS DEL EXPERTO

- ✦ Usa interfaces para tipar los parámetros de funciones complejas; tu código gana claridad inmediata.

- ✦ El modificador readonly previene mutaciones accidentales en propiedades críticas como IDs.
- ✦ Si dos interfaces comparten propiedades, considera crear una base y usar extends en lugar de duplicar.



PRACTICA LO APRENDIDO



RETO 1

Modelo de Vehículo

Crea una interface Vehiculo con placa (readonly), marca, modelo, año, kilometraje y disponible. Luego crea tres objetos de tipo Vehiculo y muéstralos en consola con un mensaje formateado por cada uno indicando su estado de disponibilidad.



RETO 2

Producto con detalles opcionales

Crea una interface Producto con id, nombre, precio (obligatorios), descuento y descripcion (opcionales). Crea dos productos: uno con todos los campos y otro solo con los obligatorios. Imprime cada uno mostrando el precio final aplicando el descuento si existe.



LECCIÓN 02

TUTORIAL TYPESCRIPT — PASO A PASO

RECURSIVIDAD EN TYPESCRIPT

Funciones que se llaman a sí mismas



CONCEPTO

La recursividad es una técnica de programación donde una función se invoca a sí misma para resolver un problema dividiéndolo en subproblemas más pequeños del mismo tipo. Toda función recursiva tiene dos partes: un caso base (que detiene las llamadas) y un caso recursivo (que reduce el problema).



EXPLICACIÓN DETALLADA

Cuando una función se llama a sí misma sin un caso base, entra en un bucle infinito que termina agotando la memoria, lo que se conoce como desbordamiento de pila o stack overflow. Por eso, definir bien el caso base es lo primero.

En cada llamada recursiva, JavaScript apila un nuevo contexto de ejecución con sus propias variables. Cuando la función llega al caso base, comienza a resolver y a desapilar cada llamada en orden inverso hasta devolver el resultado final.

La recursividad brilla en problemas con estructura natural recursiva: cálculos matemáticos como factorial o Fibonacci, recorrido de estructuras de árbol, navegación de directorios o cualquier estructura donde cada elemento contiene otros del mismo tipo.



EJEMPLO PRÁCTICO



Caso: Calcular el factorial de un número para una calculadora científica.



CÓDIGO DE EJEMPLO

```
// Calculadora científica - función factorial recursiva
function factorial(n: number): number {
  // Caso base
  if (n <= 1) return 1;
  // Caso recursivo
  return n * factorial(n - 1);
}
```

```
console.log("5! =", factorial(5));
console.log("7! =", factorial(7));
console.log("10! =", factorial(10));

// Otro ejemplo: suma de los números de 1 a n
function sumarHasta(n: number): number {
  if (n === 0) return 0;
  return n + sumarHasta(n - 1);
}

console.log("Suma 1 a 10:", sumarHasta(10));

// Fibonacci clásico
function fibonacci(n: number): number {
  if (n < 2) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
}

console.log("Fibonacci(8):", fibonacci(8));
```

▶ **SALIDA EN CONSOLA**

```
5! = 120
7! = 5040
10! = 3628800
Suma 1 a 10: 55
Fibonacci(8): 21
```

★ **TIPS DEL EXPERTO**

- ✦ Siempre define el caso base ANTES del caso recursivo: si falla ese chequeo, tu programa colapsa.
- ✦ Cada problema recursivo se puede resolver con un ciclo (iterativo). Usa recursividad cuando aclare la lógica, no por moda.
- ✦ Para entradas grandes, considera la memoización: guardar resultados ya calculados para evitar repetir trabajo.

🚀 **PRACTICA LO APRENDIDO**



Potencia recursiva

RETO 1

Crea una función recursiva potencia(base, exponente) que calcule base elevado a la exponente sin usar Math.pow. Caso base: exponente igual a 0 devuelve 1. Caso recursivo: base por potencia(base, exponente - 1). Pruébala con tres valores diferentes.



RETO 2

Cuenta regresiva recursiva

Crea una función recursiva cuentaRegresiva(n) que imprima los números desde n hasta 1, y al llegar a 0 imprima ¡Despegue!. No uses ningún ciclo (for, while). Pruébala con n igual a 10.



LECCIÓN 03

TUTORIAL TYPESCRIPT — PASO A PASO

MANEJO DE EXCEPCIONES

Controla los errores antes de que rompan tu programa

**CONCEPTO**

Una excepción es un error que ocurre durante la ejecución del programa y que, si no se maneja, hace que el programa se detenga abruptamente. TypeScript permite manejar excepciones con la estructura `try / catch / finally` para reaccionar de forma controlada cuando algo sale mal.

**EXPLICACIÓN DETALLADA**

El bloque `try` contiene el código que podría lanzar una excepción. Si ocurre un error dentro de `try`, la ejecución salta inmediatamente al bloque `catch`, que recibe el objeto del error como parámetro y permite reaccionar (mostrar un mensaje, registrar el error, intentar una alternativa).

El bloque `finally` es opcional y se ejecuta siempre, ocurra o no un error. Es ideal para tareas de limpieza como cerrar archivos, liberar recursos o reiniciar variables.

También puedes lanzar tus propios errores con la palabra `throw` seguida de un `new Error('mensaje')`. Esto es muy útil para validar parámetros y comunicar problemas específicos del negocio. En TypeScript moderno se tipa el error en `catch` con `(e: unknown)` y se verifica su tipo antes de usarlo.

**EJEMPLO PRÁCTICO**

Caso: Divisor seguro de una calculadora: evita el error de dividir entre cero.

**CÓDIGO DE EJEMPLO**

```
// Calculadora segura con manejo de errores
function dividir(a: number, b: number): number {
  if (b === 0) {
    throw new Error("No se puede dividir entre cero");
  }
  return a / b;
}
```

```
// Caso 1: división exitosa
try {
  const resultado = dividir(10, 2);
  console.log("10 / 2 =", resultado);
} catch (error: unknown) {
  if (error instanceof Error) {
    console.log("❌ Error:", error.message);
  }
}

// Caso 2: división con error
try {
  const resultado = dividir(10, 0);
  console.log("Resultado:", resultado);
} catch (error: unknown) {
  if (error instanceof Error) {
    console.log("❌ Error capturado:", error.message);
  }
} finally {
  console.log("🔄 Operación finalizada.");
}

// Validación de edad
function validarEdad(edad: number): string {
  if (edad < 0) throw new Error("Edad no puede ser negativa");
  if (edad > 130) throw new Error("Edad poco realista");
  return "Edad válida: " + edad;
}

try {
  console.log(validarEdad(-5));
} catch (e: unknown) {
  if (e instanceof Error) console.log("⚠️ ", e.message);
}
```

▶ **SALIDA EN CONSOLA**

```
10 / 2 = 5
❌ Error capturado: No se puede dividir entre cero
🔄 Operación finalizada.
⚠️ Edad no puede ser negativa
```

★ **TIPS DEL EXPERTO**

- ✦ Nunca atrapes errores en silencio con catch vacío: registra al menos el mensaje, sino estarás depurando a ciegas.
- ✦ Lanza errores con mensajes claros y específicos. 'Error 1' no le dice nada al desarrollador que reciba el problema.
- ✦ En TypeScript moderno tipa el error como unknown y verifica con instanceof Error antes de acceder a sus propiedades.



PRACTICA LO APRENDIDO



RETO 1

Cajero con validación de saldo

Crea una función retirar(saldo, monto) que devuelva el nuevo saldo. Debe lanzar una excepción si el monto es negativo o si supera el saldo disponible. Usa try/catch para hacer tres retiros: uno válido, uno con monto negativo y uno mayor al saldo.



RETO 2

Login con validaciones múltiples

Crea una función login(usuario, contraseña) que lance distintos errores: si el usuario está vacío, si la contraseña tiene menos de 6 caracteres, o si la combinación no es válida (compara contra valores guardados). Pruébala con tres escenarios diferentes.



LECCIÓN 04

TUTORIAL TYPESCRIPT — PASO A PASO

ARRAYS EN PROFUNDIDAD

Domina la estructura más usada de la programación



CONCEPTO

Un array (o arreglo) es una estructura de datos ordenada que almacena múltiples valores del mismo tipo en una sola variable, accesibles mediante un índice numérico que empieza en 0. En TypeScript se tipan con `tipo[]` o con la sintaxis `Array<tipo>`, y vienen con decenas de métodos integrados muy poderosos.



EXPLICACIÓN DETALLADA

Los arrays son objetos en JavaScript, lo que significa que tienen propiedades y métodos. La propiedad `length` devuelve la cantidad de elementos. Los índices van desde 0 hasta `length - 1`. Si intentas acceder a un índice fuera de rango obtienes `undefined`, no un error.

Existen métodos que MODIFICAN el array original (`push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`) y métodos que devuelven un NUEVO array sin modificar el original (`slice`, `concat`, `filter`, `map`). Esta distinción es vital: mezclarlos sin entender produce bugs sutiles.

TypeScript te permite tipar arrays con uniones (`(number | string)[]`), arrays anidados (`number[][]`) o tuplas con tipos fijos en cada posición [`string`, `number`]. Aprovecha estas herramientas para modelar datos con precisión.



EJEMPLO PRÁCTICO



Caso: Gestión del menú diario de un restaurante de comida rápida.



CÓDIGO DE EJEMPLO

```
// Menú del día - Restaurante Sabor Express
const menu: string[] = ["Hamburguesa", "Pizza", "Ensalada"];

// Agregar al final / al inicio
menu.push("Tacos"); // ahora son 4
menu.unshift("Sopa del día"); // ahora son 5
console.log("Menú actual:", menu);
```

```
// Quitar el último / el primero
const ultimo = menu.pop();      // quita Tacos
const primero = menu.shift();   // quita Sopa
console.log("Quitados:", primero, "y", ultimo);
console.log("Menú quedó:", menu);

// Buscar elementos
console.log("¿Hay Pizza?:", menu.includes("Pizza"));
console.log("Índice Ensalada:", menu.indexOf("Ensalada"));

// Cortar / copiar (sin modificar el original)
const copia = menu.slice(0, 2);
console.log("Primeros 2:", copia);
console.log("Original intacto:", menu);

// Ordenar
const ordenado = [...menu].sort();
console.log("Ordenado:", ordenado);

// Unir con un separador
console.log("Como texto:", menu.join(" | "));
```

SALIDA EN CONSOLA

```
Menú actual: [ 'Sopa del día', 'Hamburguesa', 'Pizza', 'Ensalada', 'Tacos' ]
Quitados: Sopa del día y Tacos
Menú quedó: [ 'Hamburguesa', 'Pizza', 'Ensalada' ]
¿Hay Pizza?: true
Índice Ensalada: 2
Primeros 2: [ 'Hamburguesa', 'Pizza' ]
Original intacto: [ 'Hamburguesa', 'Pizza', 'Ensalada' ]
Como texto: Hamburguesa | Pizza | Ensalada
```

★ TIPS DEL EXPERTO

- ✦ Para no mutar un array y conservar el original, usa el operador spread: `const copia = [...original]`.
- ✦ `splice` modifica y devuelve los elementos removidos; `slice` no modifica y devuelve una copia. ¡No los confundas!
- ✦ `sort` por defecto convierte todo a string. Para números, usa: `arr.sort((a, b) => a - b)`.



PRACTICA LO APRENDIDO



RETO 1

Carrito de compras dinámico

Crea un array de strings vacío llamado carrito. Agrega tres productos con push, agrega uno al inicio con unshift, quita el último con pop, verifica con includes si un producto está, y finalmente muestra el total de productos.



RETO 2

Ranking de jugadores

Tienes un array con los puntajes de 8 jugadores. Ordénalo de mayor a menor (sin modificar el original), obtén los 3 mejores con slice, calcula la suma de esos 3 y muestra el promedio del top 3.



LECCIÓN 05

TUTORIAL TYPESCRIPT — PASO A PASO

ESTRUCTURA MAP

Diccionarios clave-valor con tipos seguros



CONCEPTO

Map es una estructura de datos que almacena pares de clave-valor, donde tanto la clave como el valor pueden ser de cualquier tipo (no solo strings, a diferencia de los objetos planos). Mantiene el orden de inserción y proporciona operaciones muy eficientes para agregar, obtener, verificar y eliminar entradas.



EXPLICACIÓN DETALLADA

Se crea con `new Map<TipoClave, TipoValor>()`. Los métodos principales son `set(clave, valor)` para agregar o actualizar, `get(clave)` para obtener un valor, `has(clave)` para verificar existencia, `delete(clave)` para eliminar, y `clear()` para vaciar el Map por completo.

La propiedad `size` devuelve cuántos pares hay almacenados, a diferencia de los objetos planos donde necesitas `Object.keys(obj).length`. Puedes recorrer un Map con `forEach` o con `for...of` usando `map.entries()`, `map.keys()` o `map.values()`.

Map es superior a un objeto plano cuando: necesitas claves que no sean strings (objetos, números), te importa el orden de inserción, vas a agregar y eliminar entradas frecuentemente, o quieres saber con facilidad cuántas entradas hay.



EJEMPLO PRÁCTICO



Caso: Inventario de una bodega con códigos únicos de producto.



CÓDIGO DE EJEMPLO

```
// Inventario bodega - estructura Map
const inventario = new Map<string, number>();

// Agregar entradas
inventario.set("PROD-001", 50);
inventario.set("PROD-002", 120);
inventario.set("PROD-003", 30);
inventario.set("PROD-004", 200);
```

```
console.log("Total productos:", inventario.size);

// Obtener un valor
console.log("Stock PROD-002:", inventario.get("PROD-002"));

// Verificar existencia
console.log("¿Existe PROD-099?:", inventario.has("PROD-099"));

// Actualizar (mismo set sobre clave existente)
inventario.set("PROD-001", 45);
console.log("Nuevo stock PROD-001:", inventario.get("PROD-001"));

// Recorrer todas las entradas
console.log("\n=== INVENTARIO COMPLETO ===");
inventario.forEach((cantidad, codigo) => {
  console.log(`${codigo} → ${cantidad} unidades`);
});

// Eliminar
inventario.delete("PROD-003");
console.log("\nDespués de eliminar:", inventario.size);
```

▶ SALIDA EN CONSOLA

```
Total productos: 4
Stock PROD-002: 120
¿Existe PROD-099?: false
Nuevo stock PROD-001: 45

=== INVENTARIO COMPLETO ===
PROD-001 → 45 unidades
PROD-002 → 120 unidades
PROD-003 → 30 unidades
PROD-004 → 200 unidades

Después de eliminar: 3
```

★ TIPS DEL EXPERTO

- ✦ Si `.get()` devuelve `undefined`, puede que la clave no exista. Siempre verifica con `.has()` o asigna un valor por defecto.
- ✦ Map mantiene el orden de inserción. Si necesitas que las claves estén ordenadas, ordénalas tú mismo antes de iterar.

✦ Para convertir un Map a array usa `Array.from(map.entries())`. Para volver, `new Map(array)`.

PRACTICA LO APRENDIDO



Agenda telefónica

Crea un `Map<string, string>` donde la clave es el nombre y el valor el teléfono. Agrega cinco contactos. Busca y muestra el teléfono de dos contactos específicos. Elimina uno. Recorre y muestra todos los contactos restantes.



Contador de palabras

Dada una frase, crea un `Map<string, number>` que cuente cuántas veces aparece cada palabra. Recorre la frase, separa por espacios y va sumando en el Map. Al final, muestra el Map completo con cada palabra y su frecuencia.



LECCIÓN 06

TUTORIAL TYPESCRIPT — PASO A PASO

ESTRUCTURA SET

Colecciones de elementos únicos sin repetición



CONCEPTO

Set es una estructura que almacena valores ÚNICOS de cualquier tipo: si intentas agregar un valor que ya existe, simplemente se ignora. No tiene índices: el orden interno es el de inserción y se accede a los elementos mediante iteración, no por posición.



EXPLICACIÓN DETALLADA

Se crea con `new Set<Tipo>()` o pasando un array para iniciar: `new Set<number>([1, 2, 3, 2])` crea un Set con tres elementos porque el segundo 2 se descarta automáticamente. Los métodos clave son `add(valor)`, `has(valor)`, `delete(valor)`, `clear()` y la propiedad `size`.

Set es ideal cuando lo único que importa es la presencia o ausencia de un elemento, sin importar cuántas veces se intente agregar. Por ejemplo: usuarios conectados, tags de un post, elementos seleccionados en una interfaz, palabras únicas en un texto.

Una operación muy común con Set es eliminar duplicados de un array: `const sinDuplicados = [...new Set(arregloOriginal)]`. Esto convierte el array a Set (que filtra duplicados automáticamente) y de vuelta a array con el spread.



EJEMPLO PRÁCTICO



Caso: Lista de invitados a un evento sin permitir personas repetidas.



CÓDIGO DE EJEMPLO

```
// Lista de invitados - Evento Lanzamiento 2026
const invitados = new Set<string>();

invitados.add("Ana");
invitados.add("Bruno");
invitados.add("Carla");
invitados.add("Ana");           // se ignora, ya está
invitados.add("David");
```

```
console.log("Total invitados:", invitados.size);
console.log("¿Asiste Ana?:", invitados.has("Ana"));
console.log("¿Asiste Pedro?:", invitados.has("Pedro"));

// Recorrer
console.log("\n=== LISTA ===");
invitados.forEach(nombre => console.log("✓", nombre));

// Eliminar
invitados.delete("Bruno");
console.log("\nDespués de cancelación:", invitados.size);

// Eliminar duplicados de un array
const numeros: number[] = [1, 2, 3, 2, 4, 1, 5, 3];
const unicos: number[] = [...new Set(numeros)];
console.log("\nOriginal:", numeros);
console.log("Únicos: ", unicos);
```

SALIDA EN CONSOLA

```
Total invitados: 4
¿Asiste Ana?: true
¿Asiste Pedro?: false

=== LISTA ===
✓ Ana
✓ Bruno
✓ Carla
✓ David

Después de cancelación: 3

Original: [
  1, 2, 3, 2,
  4, 1, 5, 3
]
Únicos:  [ 1, 2, 3, 4, 5 ]
```

★ TIPS DEL EXPERTO

- ✦ El truco [...new Set(array)] es la forma más limpia y rápida de eliminar duplicados de un array.

- ✦ Set NO sirve para objetos con misma forma: dos objetos `{a:1}` y `{a:1}` son referencias diferentes, no duplicados.
- ✦ Si necesitas operaciones de conjunto (unión, intersección), recórrelos manualmente: Set no las trae integradas.

PRACTICA LO APRENDIDO



Sistema de etiquetas

Crea un Set para las etiquetas de un artículo de blog. Intenta agregar 'tecnología', 'IA', 'tecnología', 'desarrollo', 'IA'. Imprime el tamaño y los elementos. Verifica si tiene la etiqueta 'IA'. Elimina 'desarrollo' y muestra el Set final.



Países visitados únicos

Tienes un array con países que un viajero visitó este año (puede haber repeticiones). Crea un Set para obtener solo los países únicos. Muestra cuántos países distintos visitó y la lista ordenada alfabéticamente.



LECCIÓN 07

TUTORIAL TYPESCRIPT — PASO A PASO

MÉTODO FILTER

Extrae subconjuntos de un array según una condición



CONCEPTO

filter es un método de los arrays que devuelve un NUEVO array conteniendo únicamente los elementos del array original que cumplen una condición. La condición se expresa mediante una función callback que recibe cada elemento y debe devolver true (incluir) o false (excluir).



EXPLICACIÓN DETALLADA

La firma típica es `arr.filter((elemento, indice, array) => boolean)`. El elemento es lo más usado, el índice y el array completo son opcionales y rara vez se necesitan. La función callback debe ser pura: dada la misma entrada, devuelve el mismo resultado, sin efectos secundarios.

filter NO modifica el array original; crea uno nuevo. Si ningún elemento cumple la condición, devuelve un array vacío. Esto es muy útil porque permite encadenar operaciones sin temer mutaciones inesperadas.

Es muy común combinar filter con otros métodos: `.filter(...).map(...).reduce(...)` es el patrón canónico para transformar y resumir datos en programación funcional. Aprende este flujo y lo usarás miles de veces en tu carrera.



EJEMPLO PRÁCTICO



Caso: Filtrar empleados activos con salario alto en un sistema de RRHH.



CÓDIGO DE EJEMPLO

```
// Sistema RRHH - filtros sobre empleados
interface Empleado {
  nombre: string;
  salario: number;
  activo: boolean;
  departamento: string;
}
```

```

const empleados: Empleado[] = [
  { nombre: "Ana",      salario: 2500000, activo: true,  departamento: "Ventas"
},
  { nombre: "Bruno",    salario: 1800000, activo: false, departamento: "TI" },
  { nombre: "Carla",    salario: 3200000, activo: true,  departamento: "TI" },
  { nombre: "David",    salario: 2100000, activo: true,  departamento: "Ventas"
},
  { nombre: "Elena",    salario: 4500000, activo: true,  departamento:
"Gerencia" }
];

// Empleados activos
const activos = empleados.filter(e => e.activo);
console.log("Activos:", activos.length);

// Salario mayor a 2.5M
const salariosAltos = empleados.filter(e => e.salario > 2500000);
salariosAltos.forEach(e =>
  console.log(`💰 ${e.nombre}: ${e.salario}`)
);

// Combinado: activos de TI
const tiActivos = empleados.filter(
  e => e.activo && e.departamento === "TI"
);
console.log("\nTI activos:", tiActivos.map(e => e.nombre));

```

▶ SALIDA EN CONSOLA

```

Activos: 4
💰 Carla: $3200000
💰 Elena: $4500000

TI activos: [ 'Carla' ]

```

★ TIPS DEL EXPERTO

- ✦ Si solo necesitas el primer elemento que cumpla la condición, usa `find` en lugar de `filter` (es más eficiente).
- ✦ `filter` siempre devuelve un array, incluso si está vacío. Nunca devuelve `undefined`.
- ✦ Para condiciones complejas, extrae la lógica a una función con nombre:
`empleados.filter(esElegible)` lee mejor.



PRACTICA LO APRENDIDO



RETO 1

Productos en oferta

Tienes un array de productos con nombre, precio y descuento. Filtra los productos con descuento mayor a 20%. Para cada uno, calcula el precio final aplicando el descuento y muéstralo formateado.



RETO 2

Estudiantes destacados

Tienes un array de estudiantes con nombre, promedio y asistencia. Filtra los que tienen promedio mayor o igual a 4.0 Y asistencia mayor a 90%. Muestra cuántos cumplen y sus nombres separados por coma.



LECCIÓN 08

TUTORIAL TYPESCRIPT — PASO A PASO

MÉTODO MAP

Transforma cada elemento de un array



CONCEPTO

El método map recorre un array y devuelve un NUEVO array con la misma cantidad de elementos, donde cada elemento ha sido transformado por una función callback. Es la operación fundamental para transformar colecciones de datos sin alterar el original.



EXPLICACIÓN DETALLADA

La firma es `arr.map((elemento, indice, array) => nuevoValor)`. La función callback recibe cada elemento y debe RETORNAR el nuevo valor. A diferencia de `forEach`, map sí espera que devuelvas algo; si no lo haces, obtendrás un array de `undefined`.

El array resultante tiene exactamente el mismo tamaño que el original. Esto distingue map de filter: filter cambia el tamaño, map cambia el contenido. Si necesitas hacer ambos, encadénalos: `arr.filter(...).map(...)`.

map es ideal para extraer una propiedad de objetos (`productos.map(p => p.nombre)`), para aplicar cálculos (`numeros.map(n => n * 2)`) o para construir representaciones formateadas (`usuarios.map(u => `${u.nombre} <${u.email}>`)`).



EJEMPLO PRÁCTICO



Caso: Convertir precios en dólares a pesos colombianos en un catálogo.



CÓDIGO DE EJEMPLO

```
// Catálogo - conversión USD a COP
const precios: number[] = [10, 25, 50, 100, 200];
const tasa: number = 4100;

// Convertir cada precio
const enPesos: number[] = precios.map(p => p * tasa);
console.log("USD:", precios);
console.log("COP:", enPesos);
```

```
// Aplicar IVA del 19% a una lista
const conIVA: number[] = enPesos.map(p => p * 1.19);
console.log("Con IVA:", conIVA);

// Transformar objetos
interface Producto {
  nombre: string;
  precio: number;
}

const productos: Producto[] = [
  { nombre: "Mouse", precio: 50000 },
  { nombre: "Teclado", precio: 120000 },
  { nombre: "Monitor", precio: 800000 }
];

// Solo nombres
const nombres: string[] = productos.map(p => p.nombre);
console.log("\nProductos:", nombres);

// Etiquetas formateadas
const etiquetas: string[] = productos.map(
  p => `${p.nombre.toUpperCase()} - ${p.precio}`
);
etiquetas.forEach(e => console.log(e));
```

SALIDA EN CONSOLA

```
USD: [ 10, 25, 50, 100, 200 ]
COP: [ 41000, 102500, 205000, 410000, 820000 ]
Con IVA: [
  48790,
  121975,
  243950,
  487900,
  975800
]

Productos: [ 'Mouse', 'Teclado', 'Monitor' ]
MOUSE - $50000
TECLADO - $120000
MONITOR - $800000
```

TIPS DEL EXPERTO

- ✦ Cada vez que veas un bucle que crea otro array del mismo tamaño, refactorízalo con map: tu código gana legibilidad.
- ✦ map espera un retorno. Si tu lógica no retorna nada, lo que necesitas es forEach, no map.
- ✦ Encadena map con filter: primero filtra lo que NO te sirve, luego transforma lo que sí. Más rápido que al revés.



PRACTICA LO APRENDIDO



RETO 1

Convertidor de temperaturas

Tienes un array con temperaturas en grados Celsius. Usa map para crear un nuevo array con esas mismas temperaturas convertidas a Fahrenheit con la fórmula $F = C \times 9/5 + 32$. Muestra ambos arrays.



RETO 2

Tarjetas de empleados

Dado un array de objetos empleado con nombre, cargo y salario, usa map para generar un nuevo array de strings con el formato ' Nombre - Cargo - \$Salario'. Imprime cada tarjeta resultante.



LECCIÓN 09

TUTORIAL TYPESCRIPT — PASO A PASO

MÉTODO FOREACH

Recorre un array ejecutando una acción por elemento



CONCEPTO

forEach es un método que ejecuta una función callback por cada elemento del array. A diferencia de map o filter, forEach NO devuelve un nuevo array: su propósito es producir efectos secundarios como imprimir, modificar variables externas o invocar funciones. Devuelve undefined.



EXPLICACIÓN DETALLADA

La firma es `arr.forEach((elemento, indice, array) => void)`. El callback se ejecuta una vez por elemento. Como no devuelve nada, no se puede encadenar con otros métodos como map o filter. Si necesitas encadenar, no uses forEach.

forEach es la elección ideal cuando lo único que quieres es HACER algo con cada elemento: registrar en logs, enviar a una API, actualizar el DOM, escribir en archivo. Es decir, cuando te importa el efecto, no el resultado.

Una limitación importante: no puedes detener un forEach con break ni con return temprano (return solo sale de esa iteración, no del bucle completo). Si necesitas interrumpir el recorrido, usa for...of o el método some.



EJEMPLO PRÁCTICO



Caso: Imprimir el reporte de ventas mensuales línea por línea.



CÓDIGO DE EJEMPLO

```
// Reporte mensual - Ventas por categoría
interface Venta {
  categoria: string;
  monto: number;
}

const ventas: Venta[] = [
  { categoria: "Tecnología", monto: 4500000 },
```

```

    { categoria: "Hogar",      monto: 2300000 },
    { categoria: "Ropa",      monto: 1800000 },
    { categoria: "Alimentos", monto: 3200000 }
  ];

  console.log("=== REPORTE DE VENTAS ===");
  ventas.forEach((v, i) => {
    console.log(`${i + 1}. ${v.categoria}: ${v.monto}`);
  });

  // Acumulación con variable externa
  let totalGeneral: number = 0;
  ventas.forEach(v => {
    totalGeneral += v.monto;
  });
  console.log("-----");
  console.log("TOTAL:", totalGeneral);

  // Procesamiento múltiple
  console.log("\n=== ALERTA DE METAS ===");
  ventas.forEach(v => {
    if (v.monto > 3000000) {
      console.log(`🏆 ${v.categoria}: META SUPERADA`);
    } else {
      console.log(`⚠️ ${v.categoria}: Por debajo de meta`);
    }
  });
};

```

▶ SALIDA EN CONSOLA

```

=== REPORTE DE VENTAS ===
1. Tecnología: $4500000
2. Hogar: $2300000
3. Ropa: $1800000
4. Alimentos: $3200000
-----
TOTAL: 11800000

=== ALERTA DE METAS ===
🏆 Tecnología: META SUPERADA
⚠️ Hogar: Por debajo de meta
⚠️ Ropa: Por debajo de meta
🏆 Alimentos: META SUPERADA

```

★ TIPS DEL EXPERTO

- ✦ Si necesitas un nuevo array, NO uses `forEach`: usa `map`. `forEach` es solo para efectos secundarios.
- ✦ `forEach` no se puede romper con `break`. Si necesitas terminar antes, usa `for...of` o evalúa con `some`.
- ✦ Cada vez que escribas `forEach` para sumar/contar/agrupar, considera si `reduce` sería más claro.

🚀 PRACTICA LO APRENDIDO



RETO 1

Notas con estado

Tienes un array con las notas finales de 6 estudiantes. Usa `forEach` para imprimir cada nota junto a su número de estudiante y un mensaje: APROBADO si es 3.0 o más, REPROBADO en otro caso.



RETO 2

Bitácora de eventos

Tienes un array de objetos evento con descripción y prioridad (alta, media, baja). Usa `forEach` para imprimir cada evento con un emoji diferente según la prioridad (🔴 alta, 🟡 media, 🟢 baja) y la descripción.



LECCIÓN 10

TUTORIAL TYPESCRIPT — PASO A PASO

MÉTODO REDUCE

Combina todos los elementos en un único resultado

**CONCEPTO**

reduce es probablemente el método más poderoso de los arrays. Recorre el array y combina todos sus elementos en un ÚNICO valor (que puede ser número, string, objeto o incluso otro array). Recibe una función reductora y un valor inicial para el acumulador.

**EXPLICACIÓN DETALLADA**

La firma es `arr.reduce((acumulador, elemento, indice, array) => nuevoAcumulador, valorInicial)`. En cada iteración, la función recibe el acumulador actual y el elemento, y debe devolver el nuevo valor del acumulador. Al final, el acumulador es el resultado.

El valor inicial es CRUCIAL. Si no lo pasas, JavaScript usa el primer elemento como acumulador y empieza desde el segundo, lo cual puede causar errores si el array está vacío o si los tipos no coinciden. Siempre pasa un valor inicial explícito.

reduce puede reemplazar a la mayoría de los demás métodos: suma (acumulador + actual), producto, máximo, agrupación por propiedad, conversión de array a objeto. Aprenderlo bien te abre la puerta a la programación funcional avanzada.

**EJEMPLO PRÁCTICO**

Caso: Calcular el total de un carrito de compras y agrupar gastos por categoría.

**CÓDIGO DE EJEMPLO**

```
// Carrito de compras - cálculos con reduce
interface Item {
  nombre: string;
  precio: number;
  cantidad: number;
  categoria: string;
}

const carrito: Item[] = [
  { nombre: "Camisa", precio: 60000, cantidad: 2, categoria: "Ropa" },
```

```

    { nombre: "Zapatos", precio: 150000, cantidad: 1, categoria: "Ropa" },
    { nombre: "Pan",      precio: 4500, cantidad: 3, categoria: "Alimentos" },
    { nombre: "Leche",    precio: 5200, cantidad: 2, categoria: "Alimentos" }
  ];

  // 1. Total a pagar
  const total: number = carrito.reduce(
    (acc, item) => acc + item.precio * item.cantidad, 0
  );
  console.log("TOTAL CARRITO: $", total);

  // 2. Cantidad total de productos
  const totalItems: number = carrito.reduce(
    (acc, item) => acc + item.cantidad, 0
  );
  console.log("Items totales:", totalItems);

  // 3. Producto más caro
  const masCaro = carrito.reduce(
    (max, item) => item.precio > max.precio ? item : max
  );
  console.log("Más caro:", masCaro.nombre, "$", masCaro.precio);

  // 4. Agrupar gastos por categoría
  const porCategoría = carrito.reduce<Record<string, number>>>(
    (acc, item) => {
      const subtotal = item.precio * item.cantidad;
      acc[item.categoria] = (acc[item.categoria] || 0) + subtotal;
      return acc;
    },
    {}
  );
  console.log("Por categoría:", porCategoría);

```

▶ SALIDA EN CONSOLA

```

TOTAL CARRITO: $ 293900
Items totales: 8
Más caro: Zapatos $ 150000
Por categoría: { Ropa: 270000, Alimentos: 23900 }

```

★ TIPS DEL EXPERTO

- ✦ Siempre pasa un valor inicial. Sin él, reduce falla con arrays vacíos y produce errores difíciles de rastrear.

- ✦ El acumulador puede ser cualquier tipo: número, string, array, objeto. Esa flexibilidad es su superpoder.
- ✦ Si tu reduce se vuelve enredado, divídelo en pasos: filter + map + reduce simple. La claridad vale más que la brevedad.



PRACTICA LO APRENDIDO



RETO 1

Promedio de un curso

Tienes un array con las notas de 10 estudiantes. Usa reduce para calcular la suma total. Divide por la longitud para obtener el promedio. Muestra tanto la suma como el promedio con dos decimales.



RETO 2

Inventario total por bodega

Tienes un array de productos donde cada uno tiene nombre, bodega ('A', 'B' o 'C') y unidades. Usa reduce para crear un objeto con la suma de unidades por bodega: { A: total, B: total, C: total }. Imprime el resultado.



LECCIÓN 11

TUTORIAL TYPESCRIPT — PASO A PASO

MÉTODOS SOME Y EVERY

Pregúntale al array: ¿alguno? ¿todos?



CONCEPTO

some y every son métodos de validación que recorren un array y devuelven un booleano. some devuelve true si AL MENOS UN elemento cumple la condición; every devuelve true únicamente cuando TODOS la cumplen. Ambos cortan el recorrido en cuanto pueden decidir.



EXPLICACIÓN DETALLADA

some(callback) responde a la pregunta '¿existe al menos uno?'. Recorre el array hasta encontrar el primer elemento que cumpla la condición y entonces devuelve true. Si ninguno cumple, devuelve false. Para un array vacío, some siempre devuelve false.

every(callback) responde a '¿todos cumplen?'. Recorre el array hasta encontrar el primer elemento que NO cumpla la condición y devuelve false. Solo si todos cumplen devuelve true. Para un array vacío, every devuelve true (por convención matemática del cuantificador universal).

Son ideales para validaciones rápidas que no requieren saber cuántos cumplen ni cuáles, solo si existe alguno o si todos cumplen. Cuando uses if con un `filter.length > 0`, considera reemplazarlo por some: es más expresivo y eficiente.



EJEMPLO PRÁCTICO



Caso: Validar formularios y permisos de acceso en un sistema empresarial.



CÓDIGO DE EJEMPLO

```
// Sistema de validaciones - some y every
const edades: number[] = [25, 30, 19, 45, 38];
const requeridos: string[] = ["nombre", "email", "telefono"];
const datosFormulario: string[] = ["nombre", "email"];

// some: ¿hay algún menor de edad?
const hayMenor: boolean = edades.some(e => e < 18);
```

```
console.log("¿Hay menor de edad?:", hayMenor);

// every: ¿todos son mayores de 18?
const todosAdultos: boolean = edades.every(e => e >= 18);
console.log("¿Todos adultos?:", todosAdultos);

// Verificar formulario completo
const formularioCompleto: boolean = requeridos.every(
  campo => datosFormulario.includes(campo)
);
console.log("¿Formulario completo?:", formularioCompleto);

// Sistema de permisos
interface Usuario {
  nombre: string;
  permisos: string[];
}

const usuarios: Usuario[] = [
  { nombre: "Admin", permisos: ["leer", "escribir", "eliminar"] },
  { nombre: "Editor", permisos: ["leer", "escribir"] },
  { nombre: "Lector", permisos: ["leer"] }
];

// ¿Algún usuario puede eliminar?
const puedenEliminar = usuarios.some(
  u => u.permisos.includes("eliminar")
);
console.log("\n¿Hay quien pueda eliminar?:", puedenEliminar);

// ¿Todos pueden leer?
const todosLeen = usuarios.every(
  u => u.permisos.includes("leer")
);
console.log("¿Todos pueden leer?:", todosLeen);
```

SALIDA EN CONSOLA

```
¿Hay menor de edad?: false
¿Todos adultos?: true
¿Formulario completo?: false

¿Hay quien pueda eliminar?: true
¿Todos pueden leer?: true
```

★ TIPS DEL EXPERTO

- ♦ some y every son perezosos: cortan tan pronto pueden decidir. Mucho más eficientes que un `filter.length > 0`.
- ♦ Recuerda: every con array vacío devuelve TRUE, some con array vacío devuelve FALSE. Esto puede sorprenderte.
- ♦ Combínalos: `const validar = data.every(x => x.activo) && !data.some(x => x.bloqueado)`.

🚀 PRACTICA LO APRENDIDO



RETO 1

Validador de contraseñas

Tienes un array de contraseñas registradas en un sistema. Usa every para validar que TODAS tengan al menos 8 caracteres. Usa some para verificar si ALGUNA contiene la palabra 'admin'. Imprime ambos resultados.



RETO 2

Lote de productos vencidos

Tienes un array de productos con nombre y días_vencimiento. Usa some para detectar si hay productos con menos de 7 días para vencer. Usa every para verificar si TODOS los productos tienen más de 30 días de vida.



LECCIÓN 12

TUTORIAL TYPESCRIPT — PASO A PASO

OTROS MÉTODOS ÚTILES DE ARRAYS

Find, includes, indexOf, flat, sort y más



CONCEPTO

Además de filter, map, reduce, some y every, los arrays ofrecen otros métodos extremadamente útiles que todo desarrollador profesional debe conocer: find, findIndex, includes, indexOf, sort, reverse, flat, flatMap y join. Cada uno resuelve un problema específico de forma elegante.



EXPLICACIÓN DETALLADA

find devuelve el PRIMER elemento que cumpla una condición, o undefined si ninguno cumple. Es como filter pero solo trae uno. findIndex hace lo mismo pero devuelve el índice en lugar del elemento. Ambos cortan en el primer match.

includes verifica si un valor está presente en el array (devuelve booleano). indexOf devuelve el índice de la primera aparición, o -1 si no está. Para arrays de primitivos es perfecto; para objetos no funciona porque compara referencias.

sort ordena el array EN SITIO (modifica el original). Para números siempre pasa un comparador (a, b) => a - b. flat aplana arrays anidados un nivel; flatMap combina map y flat en una sola operación. join convierte el array en string con un separador.



EJEMPLO PRÁCTICO



Caso: Búsqueda y procesamiento avanzado de pedidos en un e-commerce.



CÓDIGO DE EJEMPLO

```
// E-commerce - operaciones avanzadas
interface Pedido {
  id: number;
  cliente: string;
  total: number;
  estado: "pendiente" | "enviado" | "entregado";
}
```

```
const pedidos: Pedido[] = [
  { id: 101, cliente: "Ana", total: 150000, estado: "entregado" },
  { id: 102, cliente: "Bruno", total: 280000, estado: "pendiente" },
  { id: 103, cliente: "Carla", total: 95000, estado: "enviado" },
  { id: 104, cliente: "David", total: 320000, estado: "pendiente" }
];

// find: primer pedido pendiente
const primerPendiente = pedidos.find(p => p.estado === "pendiente");
console.log("Pendiente:", primerPendiente?.cliente);

// findIndex: posición del pedido 103
const pos = pedidos.findIndex(p => p.id === 103);
console.log("Posición:", pos);

// includes con primitivos
const ids: number[] = pedidos.map(p => p.id);
console.log("¿Existe 102?:", ids.includes(102));

// sort: ordenar por total descendente
const ordenados = [...pedidos].sort((a, b) => b.total - a.total);
console.log("\nTop 1:", ordenados[0].cliente, "$", ordenados[0].total);

// flat: aplanar arrays anidados
const grupos: number[][] = [[1, 2], [3, 4], [5]];
console.log("Aplanado:", grupos.flat());

// flatMap: map + flat
const tags: string[][] = [["js", "ts"], ["html", "css"]];
const todos: string[] = tags.flatMap(t => t);
console.log("Todos los tags:", todos);

// join: convertir a string
const clientes: string[] = pedidos.map(p => p.cliente);
console.log("Clientes:", clientes.join(", "));
```

▶ SALIDA EN CONSOLA

```
Pendiente: Bruno
Posición: 2
¿Existe 102?: true

Top 1: David $ 320000
Aplanado: [ 1, 2, 3, 4, 5 ]
Todos los tags: [ 'js', 'ts', 'html', 'css' ]
Clientes: Ana, Bruno, Carla, David
```

★ TIPS DEL EXPERTO

- ♦ find es perfecto cuando sabes que hay UN elemento que buscas. Más rápido y expresivo que filter[0].
- ♦ sort modifica el array original. Si no quieres mutación, clona primero: [...arr].sort(...).
- ♦ Para arrays de objetos, includes no funciona como esperas. Usa some(o => o.id === buscado) en su lugar.

🚀 PRACTICA LO APRENDIDO



RETO 1

Buscador de empleados

Tienes un array de empleados con id, nombre y cargo. Usa find para encontrar al empleado con id 5. Usa findIndex para saber su posición. Usa some para verificar si existe algún 'Gerente'. Imprime los resultados.



RETO 2

Ranking ordenado

Tienes un array de jugadores con nombre y puntaje. Crea una copia ordenada de mayor a menor puntaje. Muestra el top 3 con formato '🏆 Posición. Nombre - Puntaje'. Usa join para mostrar todos los nombres separados por coma.



LECCIÓN 13

TUTORIAL TYPESCRIPT — PASO A PASO

OBJETOS EN TYPESCRIPT

Agrupa datos relacionados en una estructura



CONCEPTO

Un objeto es una estructura que agrupa propiedades (pares clave-valor) que describen una entidad. En TypeScript, los objetos se tipan especificando la forma de sus propiedades, ya sea con un type, una interface o un tipo anónimo en línea. Es el bloque fundamental para representar entidades del mundo real.



EXPLICACIÓN DETALLADA

Se declaran con llaves: { clave: valor }. Las claves son strings (aunque se escriben sin comillas si son identificadores válidos). Los valores pueden ser de cualquier tipo, incluso otros objetos o funciones. Se accede a las propiedades con punto (obj.nombre) o corchetes (obj['nombre']).

TypeScript te obliga a respetar la forma del objeto: si declaras que un objeto tiene tres propiedades, no puedes asignarle uno con menos ni con otras adicionales. Las propiedades opcionales se marcan con ? y los valores readonly previenen modificaciones.

Los objetos se pueden modificar agregando o cambiando propiedades (a menos que sean readonly o el objeto esté congelado con Object.freeze). Se pueden clonar con el spread operator ({...obj}) o convertir a JSON con JSON.stringify para serializarlos.



EJEMPLO PRÁCTICO



Caso: Modelar un usuario de una red social con sus datos y métodos.



CÓDIGO DE EJEMPLO

```
// Red social - modelo de usuario
type Usuario = {
  id: number;
  nombre: string;
  email: string;
  edad: number;
  activo: boolean;
```

```
    intereses?: string[]; // opcional
    readonly fechaRegistro: string;
};

const usuario1: Usuario = {
    id: 1,
    nombre: "Laura Mendoza",
    email: "laura@correo.com",
    edad: 28,
    activo: true,
    intereses: ["fotografía", "viajes", "música"],
    fechaRegistro: "2026-01-15"
};

// Acceso a propiedades
console.log("Nombre:", usuario1.nombre);
console.log("Email:", usuario1["email"]);
console.log("Intereses:", usuario1.intereses?.join(", "));

// Modificar
usuario1.edad = 29;
usuario1.intereses?.push("cine");

// usuario1.fechaRegistro = "x"; // ❌ Error: readonly

// Clonar con spread
const usuario2: Usuario = {
    ...usuario1,
    id: 2,
    nombre: "Mario López",
    email: "mario@correo.com"
};

console.log("\nUsuario 1:", usuario1.nombre, "→", usuario1.edad, "años");
console.log("Usuario 2:", usuario2.nombre, "→", usuario2.edad, "años");

// Object.keys / values / entries
console.log("\nClaves:", Object.keys(usuario1));
console.log("Total propiedades:", Object.keys(usuario1).length);
```

▶ SALIDA EN CONSOLA

```
Nombre: Laura Mendoza
Email: laura@correo.com
Intereses: fotografía, viajes, música
```

```
Usuario 1: Laura Mendoza → 29 años
```

Usuario 2: Mario López → 29 años

```
Claves: [  
  'id',          'nombre',  
  'email',       'edad',  
  'activo',      'intereses',  
  'fechaRegistro'  
]  
Total propiedades: 7
```

★ TIPS DEL EXPERTO

- ✦ Para clonar un objeto NO uses `const copia = original`: eso copia la referencia. Usa `{...original}` para clonar las propiedades.
- ✦ El spread es superficial: si el objeto tiene objetos anidados, esos siguen siendo referencias. Para clonado profundo usa `structuredClone(obj)`.
- ✦ `Object.freeze(obj)` bloquea TODAS las modificaciones al objeto. Útil para constantes que no deben cambiar.

🚀 PRACTICA LO APRENDIDO



RETO 1

Ficha de un libro

Crea un type Libro con: isbn (readonly), titulo, autor, paginas, año, generos (array opcional de strings). Crea dos libros, modifica el año del primero, agrega un género al segundo, y muestra ambos con todas sus propiedades.



RETO 2

Perfil profesional

Crea un objeto Perfil con nombre, profesion, experiencia (años) y habilidades (array). Crea un perfil completo. Luego clona el perfil con spread cambiándole nombre y experiencia. Imprime las claves del objeto original con `Object.keys`.



LECCIÓN 14

TUTORIAL TYPESCRIPT — PASO A PASO

ARRAYS DE OBJETOS

Listas de entidades complejas



CONCEPTO

Un array de objetos es la combinación de las dos estructuras más usadas en programación: representa una colección de entidades, cada una con múltiples propiedades. Es la forma natural de modelar listados reales como clientes, productos, transacciones, publicaciones, etc.



EXPLICACIÓN DETALLADA

Se tipan con la sintaxis `interface[]` o `type[]`. Por ejemplo, `Cliente[]` es un array donde cada elemento debe cumplir la forma de `Cliente`. TypeScript valida cada elemento al agregarlo, asegurando consistencia en toda la colección.

Sobre un array de objetos puedes aplicar todos los métodos vistos (`filter`, `map`, `reduce`, `find`, `some`, `every`). El callback recibe cada objeto completo y desde allí accedes a sus propiedades. Esta combinación es la base del procesamiento de datos en aplicaciones reales.

Para acceder a propiedades anidadas de forma segura, usa el operador de encadenamiento opcional (`?.`): `cliente?.direccion?.ciudad`. Si alguna propiedad intermedia no existe, devuelve `undefined` sin lanzar error, evitando muchos crashes en producción.



EJEMPLO PRÁCTICO



Caso: Sistema de gestión de pedidos de un restaurante con varios procesos sobre los datos.



CÓDIGO DE EJEMPLO

```
// Sistema de pedidos - Restaurante
interface Pedido {
  id: number;
  mesa: number;
  items: string[];
  total: number;
  pagado: boolean;
```

```

}

const pedidos: Pedido[] = [
  { id: 1, mesa: 3, items: ["Pizza", "Gaseosa"], total: 45000, pagado: true },
  { id: 2, mesa: 5, items: ["Hamburguesa", "Papas"], total: 28000, pagado:
false },
  { id: 3, mesa: 3, items: ["Ensalada"], total: 18000, pagado: true },
  { id: 4, mesa: 7, items: ["Pasta", "Vino"], total: 65000, pagado: false }
];

// 1. Listar todos los pedidos
console.log("=== TODOS LOS PEDIDOS ===");
pedidos.forEach(p =>
  console.log(`#${p.id} Mesa ${p.mesa} → ${p.total} ${p.pagado ? "✅" :
"⌚"}` )
);

// 2. Filtrar pendientes de pago
const pendientes = pedidos.filter(p => !p.pagado);
console.log("\nPendientes:", pendientes.length);

// 3. Total facturado (solo pagados)
const facturado = pedidos
  .filter(p => p.pagado)
  .reduce((acc, p) => acc + p.total, 0);
console.log("Facturado: $", facturado);

// 4. Buscar pedido específico
const pedido3 = pedidos.find(p => p.id === 3);
console.log("Pedido 3 mesa:", pedido3?.mesa);

// 5. Pedidos por mesa
const porMesa = pedidos.reduce<Record<number, number>>((acc, p) => {
  acc[p.mesa] = (acc[p.mesa] || 0) + 1;
  return acc;
}, {});
console.log("Pedidos por mesa:", porMesa);

```

▶ SALIDA EN CONSOLA

```

=== TODOS LOS PEDIDOS ===
#1 Mesa 3 → $45000 ✅
#2 Mesa 5 → $28000 ⌚
#3 Mesa 3 → $18000 ✅
#4 Mesa 7 → $65000 ⌚

```

```
Pendientes: 2
Facturado: $ 63000
Pedido 3 mesa: 3

Pedidos por mesa: { '3': 2, '5': 1, '7': 1 }
```

★ TIPS DEL EXPERTO

- ✦ Encadenar filter + map + reduce es el patrón estándar para reportes: filtra lo relevante, transforma y resume.
- ✦ Para buscar un objeto por ID siempre usa find, no filter[0]: es más expresivo y eficiente.
- ✦ Si tu array de objetos crece mucho y haces búsquedas frecuentes, considera convertirlo a un Map: $O(1)$ en lugar de $O(n)$.

🚀 PRACTICA LO APRENDIDO



RETO 1

Reporte de cursos

Crea un array de 5 cursos con: codigo, nombre, creditos, aprobado (boolean). Imprime todos los cursos formateados. Filtra los aprobados, calcula el total de créditos aprobados y muestra el porcentaje de cursos aprobados respecto al total.



RETO 2

Estadísticas de ventas

Crea un array de 6 ventas con: id, vendedor, monto, region (Norte/Sur/Centro). Encuentra la venta de mayor monto. Calcula el total por región usando reduce. Lista los vendedores únicos sin duplicados.



LECCIÓN 15

TUTORIAL TYPESCRIPT — PASO A PASO

QUÉ ES UNA CLASE

La plantilla para crear objetos con comportamiento



CONCEPTO

Una clase es una plantilla o molde que define las propiedades y comportamientos (métodos) que tendrán los objetos creados a partir de ella. Las clases son el corazón de la Programación Orientada a Objetos (POO) y permiten crear código reutilizable, organizado y modelado según entidades del mundo real.



EXPLICACIÓN DETALLADA

Se declara con la palabra reservada `class` seguida del nombre (por convención inicia con mayúscula). Dentro se definen las propiedades (con sus tipos) y los métodos (funciones que operan sobre esas propiedades). Una clase no es un objeto: es la receta para crearlos.

Para crear un objeto a partir de una clase se usa la palabra `new`: `const obj = new MiClase()`. Cada objeto creado con `new` es una INSTANCIA independiente, con sus propios valores en cada propiedad. Cambios en una instancia no afectan a otras.

Las clases en TypeScript ofrecen ventajas sobre los objetos planos: encapsulamiento (ocultar datos), herencia (extender funcionalidad), polimorfismo (tratar objetos diferentes con la misma interfaz) y verificación de tipos en tiempo de desarrollo. Son la base para construir aplicaciones grandes y mantenibles.



EJEMPLO PRÁCTICO



Caso: Modelar una mascota en una aplicación veterinaria.



CÓDIGO DE EJEMPLO

```
// App veterinaria - clase Mascota básica
class Mascota {
  nombre: string;
  especie: string;
  edad: number;
  vacunada: boolean;
```

```
constructor(nombre: string, especie: string, edad: number) {
    this.nombre = nombre;
    this.especie = especie;
    this.edad = edad;
    this.vacunada = false;
}

presentarse(): void {
    console.log(`Hola, soy ${this.nombre}, un ${this.especie} de
    ${this.edad} años.`);
}

vacunar(): void {
    this.vacunada = true;
    console.log(`✍️ ${this.nombre} ha sido vacunad@.`);
}

estadoVacunacion(): string {
    return this.vacunada ? "Al día" : "Pendiente";
}
}

// Crear instancias
const firulais = new Mascota("Firulais", "Perro", 3);
const michi = new Mascota("Michi", "Gato", 5);

firulais.presentarse();
michi.presentarse();

firulais.vacunar();
console.log("Firulais:", firulais.estadoVacunacion());
console.log("Michi:   ", michi.estadoVacunacion());
```

▶ SALIDA EN CONSOLA

```
Hola, soy Firulais, un Perro de 3 años.
Hola, soy Michi, un Gato de 5 años.
✍️ Firulais ha sido vacunad@.
Firulais: Al día
Michi:    Pendiente
```

★ TIPS DEL EXPERTO

- ✦ La palabra `this` dentro de una clase se refiere a la instancia actual. Sin `this`, accedes a variables globales o sales del scope.

- ✦ Una clase puede tener tantos métodos como necesites. Mantén cada método enfocado en UNA responsabilidad clara.
- ✦ Si una clase se vuelve muy grande, probablemente esté haciendo demasiado. Divídela en clases más pequeñas que colaboren entre sí.



PRACTICA LO APRENDIDO



RETO 1

Clase Libro

Crea una clase Libro con propiedades: titulo, autor, paginas, prestado. Incluye un constructor que reciba los tres primeros (prestado inicia en false). Métodos: prestar(), devolver() y estado() que imprima el estado actual. Crea dos libros y pruébalo.



RETO 2

Clase CuentaBancaria

Crea una clase CuentaBancaria con titular, saldo y numero. El constructor recibe titular y numero, el saldo inicia en 0. Métodos: depositar(monto), retirar(monto) y consultarSaldo(). Crea una cuenta, haz dos depósitos y un retiro, muestra el saldo final.



LECCIÓN 16

TUTORIAL TYPESCRIPT — PASO A PASO

CONSTRUCTOR, PROPIEDADES Y MÉTODOS

Componentes esenciales de una clase

**CONCEPTO**

Dentro de una clase encontramos tres componentes fundamentales: el constructor (método especial que se ejecuta al crear una instancia con `new` e inicializa el objeto), las propiedades (los datos que cada instancia almacena) y los métodos (funciones que definen lo que el objeto puede hacer). Adicionalmente, los modificadores de visibilidad controlan quién puede acceder a cada miembro.

**EXPLICACIÓN DETALLADA**

El constructor se declara con la palabra reservada `constructor` seguida de paréntesis con los parámetros de inicialización. Dentro, se asignan valores a las propiedades usando `this.propiedad`. Solo puede haber un constructor por clase. Si no defines uno, TypeScript crea uno vacío por defecto.

Los modificadores de visibilidad en TypeScript son: `public` (acceso desde cualquier lugar, es el valor por defecto), `private` (solo accesible desde la misma clase), `protected` (accesible desde la clase y sus subclases), y `readonly` (solo se puede asignar en la declaración o en el constructor, después es de solo lectura).

TypeScript ofrece un atajo elegante: declarar las propiedades directamente en los parámetros del constructor anteponiendo el modificador. `constructor(public nombre: string, private edad: number)` crea automáticamente las propiedades y las inicializa. Es el estilo profesional preferido en 2026.

**EJEMPLO PRÁCTICO**

Caso: Sistema de gestión de empleados con datos públicos y privados.

**CÓDIGO DE EJEMPLO**

```
// Sistema RRHH - clase Empleado con visibilidad
class Empleado {
  // Atajo: propiedades en el constructor
```

```

    constructor(
        public readonly id: number,      // pública pero inmutable
        public nombre: string,           // pública, modificable
        public cargo: string,            // pública
        private salario: number,         // privada (interna)
        protected fechaIngreso: string  // protegida (heredable)
    ) {}

    // Método público: lee propiedad privada
    presentar(): void {
        console.log(`👤 ${this.nombre} - ${this.cargo}`);
        console.log(`    Salario: ${this.salario}`);
    }

    // Método público para modificar salario
    aumentarSalario(porcentaje: number): void {
        if (porcentaje <= 0) {
            console.log("❌ Aumento inválido");
            return;
        }
        const aumento = this.salario * (porcentaje / 100);
        this.salario += aumento;
        console.log(`💰 +${porcentaje}% aplicado. Nuevo salario:
    ${this.salario}`);
    }

    // Método privado: solo uso interno
    private calcularPrima(): number {
        return this.salario * 0.05;
    }

    // Método público que usa uno privado
    pagarPrima(): void {
        const prima = this.calcularPrima();
        console.log(`🎁 Prima pagada a ${this.nombre}: ${prima}`);
    }
}

const ana = new Empleado(101, "Ana Ramírez", "Desarrolladora", 3500000, "2024-
03-15");
ana.presentar();
ana.aumentarSalario(10);
ana.pagarPrima();

// ana.salario;           // ❌ Error: privada
// ana.calcularPrima();    // ❌ Error: privada
console.log("ID:", ana.id); // ✅ pública

```

```
// ana.id = 999; // ❌ Error: readonly
```

▶ SALIDA EN CONSOLA

```
👤 Ana Ramírez - Desarrolladora  
Salario: $3500000  
💰 +10% aplicado. Nuevo salario: $3850000  
🏠 Prima pagada a Ana Ramírez: $192500  
ID: 101
```

★ TIPS DEL EXPERTO

- ✦ Usa el atajo de constructor con modificadores: te ahorra muchas líneas y deja claro el contrato de la clase.
- ✦ Marca como private todo lo que NO deba ser tocado desde fuera. Encapsular bien evita bugs en proyectos grandes.
- ✦ readonly es perfecto para IDs y datos inmutables: garantiza que nadie los modifique accidentalmente.

🚀 PRACTICA LO APRENDIDO



RETO 1

Clase Producto con encapsulamiento

Crea una clase Producto con: id (readonly), nombre (public), precio (private), stock (private). Constructor que reciba los cuatro. Métodos: aumentarPrecio(porcentaje), reducirStock(cantidad), mostrarFicha(). Crea un producto, simula una venta y muestra el resultado.



RETO 2

Clase Estudiante con notas

Crea una clase Estudiante con: codigo (readonly), nombre (public) y notas (private array). Constructor con código y nombre. Métodos: agregarNota(n), promedio(), aprobado() que retorne true si el promedio es 3.0 o más. Pruébala con cinco notas.



LECCIÓN 17

TUTORIAL TYPESCRIPT — PASO A PASO

ENCAPSULAMIENTO, GETTERS Y SETTERS

Controla cómo se lee y modifica el estado interno

**CONCEPTO**

El encapsulamiento es uno de los cuatro pilares de la POO. Consiste en proteger el estado interno de un objeto, exponiendo solo lo necesario y controlando cómo se accede y modifica. Los getters y setters son métodos especiales que parecen propiedades pero permiten interceptar la lectura y escritura para aplicar lógica.

**EXPLICACIÓN DETALLADA**

Un getter se define con la palabra `get` antes del nombre del método, no recibe parámetros y debe devolver un valor. Un setter se define con `set`, recibe un parámetro y no devuelve nada. Desde fuera, se usan como propiedades: `obj.precio` (sin paréntesis), aunque internamente sean métodos.

La combinación clásica es: propiedad privada (con guión bajo por convención, ej. `_precio`) más getter y setter públicos. El setter permite validar antes de asignar (rechazar valores inválidos, lanzar errores, transformar el valor), algo imposible con propiedades simples.

El encapsulamiento te protege en proyectos grandes: si mañana cambias la lógica interna (por ejemplo guardar precio en otra unidad), las clases que usan tu objeto NO se enteran porque siguen accediendo a `obj.precio`. Cambias la implementación sin romper el contrato.

**EJEMPLO PRÁCTICO**

Caso: Termómetro inteligente con validación de rangos y conversión de unidades.

**CÓDIGO DE EJEMPLO**

```
// Termómetro inteligente con getters y setters
class Termometro {
  private _celsius: number = 0;

  // GETTER: lectura controlada
  get celsius(): number {
    return this._celsius;
  }
}
```

```
}

// SETTER: escritura con validación
set celsius(valor: number) {
  if (valor < -273.15) {
    throw new Error("Imposible: bajo el cero absoluto");
  }
  if (valor > 5000) {
    throw new Error("Temperatura fuera de rango razonable");
  }
  this._celsius = valor;
}

// GETTER calculado: convierte automáticamente
get fahrenheit(): number {
  return this._celsius * 9 / 5 + 32;
}

set fahrenheit(valor: number) {
  this.celsius = (valor - 32) * 5 / 9;
}

get kelvin(): number {
  return this._celsius + 273.15;
}

get estado(): string {
  if (this._celsius < 0) return "❄️ Congelado";
  if (this._celsius < 25) return "☁️ Templado";
  if (this._celsius < 35) return "☀️ Cálido";
  return "🔥 Caliente";
}
}

const t = new Termometro();
t.celsius = 25;

console.log("Celsius:   ", t.celsius);
console.log("Fahrenheit:", t.fahrenheit);
console.log("Kelvin:    ", t.kelvin);
console.log("Estado:    ", t.estado);

t.fahrenheit = 100;
console.log("\nTras setear F=100:");
console.log("Celsius:", t.celsius.toFixed(2));
console.log("Estado: ", t.estado);
```

```
try {
  t.celsius = -300;
} catch (e: unknown) {
  if (e instanceof Error) console.log("\n❌", e.message);
}
```

▶ SALIDA EN CONSOLA

```
Celsius:    25
Fahrenheit: 77
Kelvin:     298.15
Estado:     ☀ Cálido
```

```
Tras setear F=100:
Celsius: 37.78
Estado:   🔥 Caliente
```

```
❌ Imposible: bajo el cero absoluto
```

★ TIPS DEL EXPERTO

- ✦ Por convención, las propiedades privadas que se exponen por get/set llevan guión bajo (_nombre) para distinguirlas.
- ✦ Usa setters para VALIDAR antes de asignar: rangos, formatos, longitudes. Es la mejor línea de defensa.
- ✦ Un getter no debería tener efectos secundarios (modificar estado, llamar APIs). Solo leer y devolver.



PRACTICA LO APRENDIDO



RETO 1

Cuenta bancaria con validación

Crea una clase Cuenta con `_saldo` privado, `_titular` privado. Crea getter saldo (solo lectura), getter/setter titular (no permite vacío). Método depositar(m) que valide monto positivo. Prueba con depósitos válidos e inválidos.



RETO 2

Estudiante con promedio calculado

Crea una clase Estudiante con `_notas` privadas (array). Getter notas (devuelve copia). Setter notas (valida que todas estén entre 0 y 5). Getter promedio

(calculado). Getter estado (devuelve APROBADO o REPROBADO según promedio).



LECCIÓN 18

TUTORIAL TYPESCRIPT — PASO A PASO

INTERFACES APLICADAS A CLASES

Contratos que las clases deben cumplir



CONCEPTO

Las interfaces no solo sirven para tipar objetos: también pueden aplicarse a clases mediante la palabra reservada `implements`. Una clase que implementa una interface se compromete a tener todas las propiedades y métodos que la interface declara. Esto fuerza un contrato explícito y permite el polimorfismo.



EXPLICACIÓN DETALLADA

La sintaxis es `class NombreClase implements NombreInterface`. Si la clase no cumple con todo lo declarado en la interface, TypeScript marca error en tiempo de compilación. Una clase puede implementar MÚLTIPLES interfaces separándolas con comas: `class A implements I1, I2, I3`.

Las interfaces para clases pueden declarar tanto propiedades como firmas de métodos (sin cuerpo). La clase debe proporcionar la implementación concreta. Esto separa el QUÉ (interface) del CÓMO (clase), un principio fundamental del diseño profesional.

Implementar interfaces permite el polimorfismo: puedes tener una variable tipada como la interface y asignarle cualquier objeto de cualquier clase que la implemente. El código que usa la variable no necesita saber qué clase concreta es, solo que cumple el contrato.



EJEMPLO PRÁCTICO



Caso: Sistema de notificaciones con múltiples canales (email, SMS, push).



CÓDIGO DE EJEMPLO

```
// Sistema de notificaciones - múltiples canales
interface Notificable {
    destinatario: string;
    enviar(mensaje: string): void;
}

interface Registrable {
```

```

    registrarEnvio(): void;
}

// Implementa UNA interface
class NotificacionEmail implements Notificable {
    constructor(public destinatario: string) {}

    enviar(mensaje: string): void {
        console.log(`✉ Email a ${this.destinatario}: ${mensaje}`);
    }
}

// Implementa DOS interfaces
class NotificacionSMS implements Notificable, Registrable {
    private envios: number = 0;

    constructor(public destinatario: string) {}

    enviar(mensaje: string): void {
        console.log(`📱 SMS a ${this.destinatario}: ${mensaje}`);
        this.envios++;
    }

    registrarEnvio(): void {
        console.log(`📱 Total envíos SMS: ${this.envios}`);
    }
}

class NotificacionPush implements Notificable {
    constructor(public destinatario: string) {}

    enviar(mensaje: string): void {
        console.log(`🔔 Push a ${this.destinatario}: ${mensaje}`);
    }
}

// Polimorfismo: array de Notificable acepta cualquier implementación
const canales: Notificable[] = [
    new NotificacionEmail("ana@correo.com"),
    new NotificacionSMS("+57300123456"),
    new NotificacionPush("device-token-abc")
];

console.log("=== ALERTA MASIVA ===");
canales.forEach(c => c.enviar("¡Tu pedido llegó!"));

```

 **SALIDA EN CONSOLA**

=== ALERTA MASIVA ===

✉ Email a ana@correo.com: ¡Tu pedido llegó!

📱 SMS a +57300123456: ¡Tu pedido llegó!

🔔 Push a device-token-abc: ¡Tu pedido llegó!

★ TIPS DEL EXPERTO

- ✦ Diseña primero la interface, después las clases. Define el QUÉ antes del CÓMO.
- ✦ Una clase puede implementar varias interfaces; piensa en cada una como una capacidad independiente.
- ✦ Cuando tipos parámetros, prefiere la interface a la clase concreta: tu código será más flexible y testeable.

🚀 PRACTICA LO APRENDIDO



RETO 1

Sistema de pagos

Crea una interface MedioPago con propiedad usuario y método pagar(monto: number). Crea tres clases que la implementen: PagoTarjeta, PagoEfectivo, PagoTransferencia. Cada una imprime un mensaje distinto. Crea un array de MedioPago y procesa pagos.



RETO 2

Vehículos contratados

Crea una interface Conducible con métodos arrancar() y detener(), y propiedad velocidadMax. Crea las clases Auto, Moto y Bicicleta que la implementen con velocidades diferentes. Crea instancias y simula arrancarlas y detenerlas.



LECCIÓN 19

TUTORIAL TYPESCRIPT — PASO A PASO

HERENCIA Y CLASES ABSTRACTAS

Comparte comportamiento entre clases relacionadas



CONCEPTO

La herencia permite que una clase (hija) extienda a otra (padre) heredando sus propiedades y métodos, y agregando o sobrescribiendo lo que necesite. Se logra con la palabra `extends`. Las clases abstractas (declaradas con `abstract`) son clases base que NO pueden instanciarse directamente, solo heredarse.



EXPLICACIÓN DETALLADA

Cuando una clase extiende a otra, hereda todo lo público y protegido del padre. El constructor de la hija debe llamar al constructor del padre con `super(...)` antes de usar `this`. Los métodos heredados pueden sobrescribirse en la hija para personalizar el comportamiento (`override`).

Las clases abstractas (`abstract class`) sirven como plantilla para otras clases. Pueden tener métodos implementados (compartidos) y métodos abstractos (sin implementación, marcados con `abstract`) que las hijas DEBEN implementar. No se pueden instanciar con `new` directamente.

La herencia es poderosa pero peligrosa si se abusa: prefiere composición (tener una propiedad de otro tipo) sobre herencia profunda. Una jerarquía con más de tres niveles suele indicar un mal diseño. Úsala cuando exista una clara relación 'es un' (un Perro ES un Animal).



EJEMPLO PRÁCTICO



Caso: Sistema de empleados de una empresa con cargos especializados.



CÓDIGO DE EJEMPLO

```
// Sistema RRHH - herencia y clase abstracta
abstract class Empleado {
  constructor(
    public nombre: string,
    protected salarioBase: number
```

```

    ) {}

    // Método compartido por todos los hijos
    presentar(): void {
        console.log(`👤 ${this.nombre} - ${this.cargo()}`);
    }

    // Método abstracto: cada hijo DEBE implementarlo
    abstract cargo(): string;
    abstract calcularSalario(): number;
}

class Desarrollador extends Empleado {
    constructor(
        nombre: string,
        salarioBase: number,
        private nivel: "Junior" | "Senior"
    ) {
        super(nombre, salarioBase);
    }

    cargo(): string {
        return `Desarrollador ${this.nivel}`;
    }

    calcularSalario(): number {
        const bono = this.nivel === "Senior" ? 1500000 : 500000;
        return this.salarioBase + bono;
    }
}

class Vendedor extends Empleado {
    constructor(
        nombre: string,
        salarioBase: number,
        private ventas: number
    ) {
        super(nombre, salarioBase);
    }

    cargo(): string {
        return "Vendedor";
    }

    calcularSalario(): number {
        const comision = this.ventas * 0.05;
        return this.salarioBase + comision;
    }
}

```

```

    }
}

// const e = new Empleado(...); // ❌ Error: abstract

const equipo: Empleado[] = [
    new Desarrollador("Ana", 3000000, "Senior"),
    new Desarrollador("Bruno", 2500000, "Junior"),
    new Vendedor("Carla", 1800000, 1200000)
];

equipo.forEach(e => {
    e.presentar();
    console.log(` 💰 Salario total: ${e.calcularSalario()}`);
});

```

▶ SALIDA EN CONSOLA

```

👤 Ana - Desarrollador Senior
💰 Salario total: $4500000
👤 Bruno - Desarrollador Junior
💰 Salario total: $3000000
👤 Carla - Vendedor
💰 Salario total: $2400000

```

★ TIPS DEL EXPERTO

- ✦ Siempre llama a `super(...)` en el primer paso del constructor hijo: sin eso, `this` no está disponible y TypeScript lanza error.
- ✦ Si te encuentras escribiendo el mismo código en varias clases hermanas, considéralo: probablemente debería estar en la clase padre.
- ✦ Una clase abstract debe tener AL MENOS un método abstract; si no, hazla concreta y normal.

🚀 PRACTICA LO APRENDIDO



RETO 1

Jerarquía de figuras

Crea una clase abstracta `Figura` con método abstracto `area()`. Crea tres hijas: `Circulo`, `Rectangulo`, `Triangulo`, cada una con sus propiedades necesarias. Cada

una implementa `area()`. Crea un array de `Figura`, recorrela e imprime el área de cada una.



RETO 2

Animales con sonidos

Crea una clase abstracta `Animal` con nombre y método abstracto `hacerSonido()`. Crea `Perro`, `Gato`, `Vaca` que la extiendan, cada uno con su sonido. Agrega un método `comer()` en la clase padre que todas hereden. Crea instancias y haz que hagan sus sonidos.



LECCIÓN 20

TUTORIAL TYPESCRIPT — PASO A PASO

GENÉRICOS EN TYPESCRIPT

Código reutilizable que funciona con cualquier tipo



CONCEPTO

Los genéricos permiten escribir funciones, clases e interfaces que trabajan con CUALQUIER tipo de dato sin perder la seguridad de tipos. En lugar de duplicar código para cada tipo (uno para números, otro para strings), usas un parámetro de tipo (típicamente T) que se resuelve al momento de usar la función o clase.



EXPLICACIÓN DETALLADA

La sintaxis es agregar `<T>` después del nombre. Ese T es un placeholder que será reemplazado por el tipo real cuando se use. Por convención T (Type), pero puedes usar K (Key), V (Value), E (Element) o nombres descriptivos. Puedes tener varios: `<T, U, V>`.

Los genéricos también se pueden restringir con `extends` para garantizar que el tipo cumpla con cierta forma. Por ejemplo `<T extends { id: number }>` obliga a que T tenga al menos una propiedad `id`. Esto combina flexibilidad con seguridad: aceptas muchos tipos, pero todos cumplen un mínimo.

Son el corazón de las estructuras de datos reutilizables: `Array<T>`, `Map<K,V>`, `Set<T>` son genéricos. Construir tus propias clases genéricas (una Cola, una Pila, un Repositorio) te permite reutilizarlas para cualquier tipo sin reescribir el código, manteniendo el tipado fuerte.



EJEMPLO PRÁCTICO



Caso: Crear una caja genérica que puede contener cualquier tipo de elemento, manteniendo el tipado.



CÓDIGO DE EJEMPLO

```
// Genéricos - estructuras reutilizables tipadas

// 1. Función genérica
function primero<T>(arr: T[]): T | undefined {
  return arr[0];
}
```

```
}

console.log(primeros<number>([10, 20, 30]));
console.log(primeros<string>(["a", "b", "c"]));

// 2. Clase genérica: una Caja contenedora
class Caja<T> {
  private contenido: T[] = [];

  agregar(item: T): void {
    this.contenido.push(item);
  }

  obtener(indice: number): T | undefined {
    return this.contenido[indice];
  }

  tamaño(): number {
    return this.contenido.length;
  }

  listar(): void {
    this.contenido.forEach((c, i) => console.log(`  ${i}: `, c));
  }
}

const cajaNumeros = new Caja<number>();
cajaNumeros.agregar(10);
cajaNumeros.agregar(20);
cajaNumeros.agregar(30);
console.log("Caja números:");
cajaNumeros.listar();

const cajaStrings = new Caja<string>();
cajaStrings.agregar("manzana");
cajaStrings.agregar("pera");
console.log("\nCaja strings:");
cajaStrings.listar();

// 3. Genérico con restricción
interface ConId { id: number; }

function buscarPorId<T extends ConId>(items: T[], id: number): T | undefined {
  return items.find(i => i.id === id);
}

const usuarios = [
```

```
{ id: 1, nombre: "Ana" },  
{ id: 2, nombre: "Bruno" }  
];  
  
const encontrado = buscarPorId(usuarios, 2);  
console.log("\nEncontrado:", encontrado?.nombre);
```

▶ SALIDA EN CONSOLA

```
10  
a  
Caja números:  
  0: 10  
  1: 20  
  2: 30  
  
Caja strings:  
  0: manzana  
  1: pera  
  
Encontrado: Bruno
```

★ TIPS DEL EXPERTO

- ♦ Empieza con genéricos simples (`<T>`) y avanza a restricciones (`<T extends ...>`) cuando los necesites realmente.
- ♦ TypeScript suele inferir el tipo automáticamente: `caja.agregar(10)` ya sabe que es `Caja<number>`. No siempre tienes que escribirlo.
- ♦ Los genéricos NO sirven para tener tipos diferentes en una misma estructura: si necesitas eso, usa una unión (`number | string`).

🚀 PRACTICA LO APRENDIDO



RETO 1

Pila genérica

Crea una clase `Pila<T>` con métodos: `apilar(item: T)`, `desapilar()` que retorne el último y lo quite, `tope()` que retorne el último sin quitarlo, y `vacia()` que retorne boolean. Prueba la pila con números y luego con strings.



RETO 2

Repositorio genérico

Crea una interface `ConId` con propiedad `id: number`. Crea una clase `Repositorio<T extends ConId>` con métodos `agregar(item: T)`, `buscarPorId(id)`, `eliminar(id)` y `todos()`. Pruébala con un array de productos y otro de usuarios, ambos con `id`.



¡LO LOGRASTE!

Has completado el tutorial paso a paso de TypeScript



LO QUE AHORA DOMINAS

- ✓ Estructuras de datos: arrays, Map, Set y objetos.
- ✓ Métodos funcionales: filter, map, reduce, some, every, find.
- ✓ Programación Orientada a Objetos: clases, herencia, abstracción.
- ✓ Encapsulamiento y modificadores de visibilidad.
- ✓ Interfaces, recursividad y manejo de excepciones.
- ✓ Genéricos para código reutilizable y fuertemente tipado.



¿QUÉ SIGUE?

- Crea un proyecto personal: una API con Node.js + Express + TypeScript.
- Aprende un framework frontend: React, Vue o Angular con TypeScript.
- Explora tipos avanzados: utility types, mapped types, conditional types.
- Practica con plataformas como TypeScript Playground y Codewars.

< / > **TYPESCRIPT MASTER** < / >

El código de calidad empieza con tipos sólidos.